EVER 2024 Autonomous Track

Team: E-Rally/Helwan University

System Overview

The integrated system is designed for object detection and autonomous vehicle control. It comprises several modules:

LINEAR TRACK, LANE CHANGE TRACK, CIRCULAR TRACK & CUSTOM TRACK:

1. **Object Detection Module**: Uses YOLO (You Only Look Once) model to detect objects such as cars, cones, and pedestrians in real-time from image data.
2. **Kalman Filter Module**: Filters noisy position and orientation data from the vehicle's odometry sensors to provide accurate state estimates.
3. **PID Controller Module**: Controls the vehicle's speed and steering angle based on the distance to detected objects and the desired path.
4. **Vehicle Control Module**: Manages the vehicle's movement, adjusting speed and steering based on outputs from the PID controller and object detection module to do the corresponding track (LINEAR TRACK, LANE CHANGE TRACK ,CIRCULAR TRACK & CUSTOM TRACK).
5. **Odometry Management Module**: Processes odometry data to update the vehicle's current position and orientation.

Methodology Used

LINEAR TRACK, LANE CHANGE TRACK, CIRCULAR TRACK & CUSTOM TRACK:

Object Detection:

- Models used:
  In **LINEAR TRACK, LANE CHANGE TRACK, CIRCULAR TRACK:**
  - Model: *YOLO model (CPC_model.pt)* is used for object detection

  But it in **CUSTOM TRACK:**
  - Model: *YOLO model* (*custom_c7_model.pt)* is used instead for object detection.

    This new model is the same as the previous model, but it was trained extensively to handle dynamic scenarios because the **custom track** is more complex, and dynamic compared to others. This required more training to ensure effective performance.

- Libraries: Utilizes torch, cv2, and rospy.
- Detection Process:
  - Converts ROS image messages to OpenCV images using CvBridge.
  - Feeds images to the YOLO model for detection.
  - Filters detections based on confidence level and known object dimensions.
  - To calibrate a camera for distance measurement, I used the following observations for a 1-meter-long object:
    - At a distance of 5 meters, the object measures 105 pixels on the screen. (5, 105)
    - At a distance of 10 meters, the object measures 52.5 pixels on the screen. (10, 52.5)

    These points form a linear relationship, which we can describe with a line equation. First, we calculate the slope (m):

$$\text{slope} = \frac{\Delta y}{\Delta x} = \frac{(52.5 - 105)}{(10 - 5)} = \frac{-52.5}{5} = -10.5$$

Next, we use one of the points to find the y-intercept (b). Using the point (5, 105) in the line equation (y = mx + b):

$$105 = (-10.5 * 5) + b \qquad \text{so,} \quad b = 105 + 52.5 = 157.5$$

Therefore, the equation of the line that relates the distance (x) to the pixel height (y) is: $y = -10.5x + 157.5$

Given that there is a linear relationship between the height of the object and the pixels on the screen, we can generalize the equation for objects of different heights. If (k) is the actual height of the object, the equation becomes: $y = (-10.5x + 157.5) * k$

$where$:

- (y) is the number of pixels on the screen representing object's height,
- (x) is the distance between the camera and the object,
- (k) is the actual height of the object.

This equation allows us to determine the pixel height of any object at a given distance, facilitating accurate distance measurement using the camera.

- Distance Estimation: Calculated based on the height of the detected object's bounding box and its known real-world height using the formula:

$$distance = max((\frac{(y2 - y1)}{(\ known_height_m[obj_name])}) - 157.5) \ / (-10.5))$$

Kalman Filter:

- Purpose: Filters the noisy **odometry** data to estimate accurate vehicle state (position and orientation).
- Prediction Step: Predicts the next state based on the current state and control input.

$$Xpredicted = A * X + B * U \qquad Ppredicted = A * P * A^T + Q$$

- Correction Step: Updates the state estimate using the Kalman gain and measurement.

$$K_{gain} = P_{predicted} * C^T * \left(C * P_{predicted} * C^T + R\right)^{-1}$$

$$Xfiltered = Xpredicted + Kgain * (Z - C * Xpredicted)$$

$$Pfiltered = Ppredicted - Kgain * C * Ppredicted$$

PID Controller:

- Purpose: Maintains the desired distance to detected objects and the desired path.
- Libraries: rospy for ROS integration.
- Control Law: $error = target_{distance} - current_{distance}$

$$cumulative_{error} += error \times \Delta t$$

$$derivative = \frac{error - last_{error}}{\Delta t}$$

$$output = Kp \times error + Ki \times cumulative_{error} + Kd \times derivative$$

Odometry Management:

- Data Processing: Subscribes to odometry data and uses Kalman filters to estimate the vehicle's current position and orientation.
- Angle Adjustment: Ensures continuous orientation measurement by accounting for angle wrapping and tracking orientation multipliers.

Tracks Description

1. **LINEAR TRACK:** follows a straight path using PID control to maintain a set distance from the start point.

```python
#!/usr/bin/env python3

import rospy, numpy as np, cv2, torch, copy
from std_msgs.msg import Float64
from nav_msgs.msg import Odometry
from tf.transformations import euler_from_quaternion
import queue
import math
from cv_bridge import CvBridge
from ultralytics import YOLO
from sensor_msgs.msg import Image

class ObjectDetection:
    def __init__(self):
        self.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

        self.clss = ["car", "cone", "pedestrian"]
        self.model = YOLO("/home/morad/Downloads/E-Rally/Milestone_1/f_ms/src/linear_track/src/CPC_model.pt")
        self.bridge = CvBridge()
        self.original_dict = {"car": [], "cone": [], "pedestrian": []}
        self.known_height_m = {"car": 1.55, "cone": 0.66, "pedestrian": 1.65}
        self.colors = [(238, 75, 43), (170, 255, 0), (0, 150, 255)]

    def detect(self, msg):
        self.detected = copy.deepcopy(self.original_dict)
        image = self.bridge.imgmsg_to_cv2(msg, "bgr8")
        results = self.model(image)
        boxes = results[0].boxes.cpu().numpy()  # Bounding boxes
        if len(boxes) > 0:
            for box in boxes:
                data = box.data[0]
                obj_name = self.clss[data[-1].astype(int)]
                confidence = data[-2].astype(float)
                x1, y1, x2, y2 = data[:4].astype(int)

                if confidence > 0.7:
                    object_in_shape = self.isInside(480, 240, 45, 480, 915, 480, x2 if x2 < 480 else x1, y2)
                    if object_in_shape:
                        distance = max((((( (y2-y1) / self.known_height_m[obj_name] ) - 157.5) / -10.5), 0)
                        self.detected[obj_name].append(distance)
                    else:
                        distance = "N/A"
                    self.plot((image, data[-1].astype(int), (x1, y1, x2, y2), distance))
                else:
                    self.plot((image, None))
        else:
            self.plot((image, None))

        if self.detected["pedestrian"]:
            vCont.pedestrian_distance.put(min(self.detected["pedestrian"]))
        else:
            vCont.pedestrian_distance.put(500)

    def plot(self, data):
        image = data[0]
        id = data[1]
        if id is not None:
            pos = data[2]
            try:
                dis = round(data[3], 2)
            except:
                dis = "N/A"
            cv2.rectangle(image, (pos[0], pos[1]), (pos[2], pos[3]), self.colors[id], 2)
            cv2.putText(image, f"dist: {dis}", (pos[0], pos[1]-5), cv2.FONT_ITALIC, 0.5, self.colors[id], 2)
        cv2.imshow("Image", image)
        cv2.waitKey(1)

    def isInside(self, x1, y1, x2, y2, x3, y3, x, y):
        # Calculate area of the main triangle
        A = abs(x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2)) / 2.0

        # Calculate area of the sub-triangles
        A1 = abs(x * (y2 - y3) + x2 * (y3 - y) + x3 * (y - y2)) / 2.0
        A2 = abs(x1 * (y - y3) + x * (y3 - y1) + x3 * (y1 - y)) / 2.0
        A3 = abs(x1 * (y2 - y) + x2 * (y - y1) + x * (y1 - y2)) / 2.0

        # Check if the point is inside the triangle
        return A == A1 + A2 + A3

class Kalman_Filter:
    def __init__(self, r1, r2, r3):
        # Define the Kalman filter parameters and variables
        self.dt = 0.1  # Time step
        self.A = np.array([[1, self.dt, 0, 0, 0, 0],
                           [0, 1, 0, 0, 0, 0],
                           [0, 0, 1, self.dt, 0, 0],
                           [0, 0, 0, 1, 0, 0],
                           [0, 0, 0, 0, 1, self.dt],
                           [0, 0, 0, 0, 0, 1]])  # State transition matrix
        self.B = np.zeros((6, 1))  # Control input matrix
        self.C = np.array([[1, 0, 0, 0, 0, 0],
                           [0, 0, 1, 0, 0, 0],
                           [0, 0, 0, 0, 1, 0]])  # Measurement matrix
        self.Q = np.eye(6) * 0.01  # Process noise covariance matrix
        self.R = np.array([[r1, 0, 0],
                           [0, r2, 0],
                           [0, 0, r3]])  # Measurement noise covariance matrix
        self.U = np.array([[0],[0.2],[0]])
        self.P = np.eye(6) * 10000  # Initial covariance matrix
        self.X = np.zeros((6, 1))  # Initial state estimate
        self.filtered_odom = []

        # Kalman filter prediction step
    def kf_prediction(self):
        self.Xpredicted = np.matmul(self.A, self.X) + np.matmul(self.B, self.U)         ##Predicted state vector                          (6x1)
        self.Ppredicted = np.matmul(np.matmul(self.A, self.P), np.transpose(self.A)) + self.Q   ##Predicted error co-variance matrix    (6x6)
        return (self.Xpredicted, self.Ppredicted)
    def kf_correction(self, Z):
        self.CTrans = np.transpose(self.C)
        self.num = np.matmul(self.Ppredicted, self.CTrans)      ##Numerature of Kalman gain equation
        self.den1 = np.matmul(self.C, self.Ppredicted)      ##CMatrix * PMatrix
        self.den2 = np.matmul(self.den1, self.CTrans) + self.R      ##CMatrix * PMatrix * CMatrix^T _+ R
        self.den = np.matrix(self.den2)                     ##Place denemrature in matrix form
        self.deninv = self.den.getI()                       ##(CMatrix * PMatrix * CMatrix^T _+ R) Inverse
        self.KGain = np.matmul(self.num, self.deninv)       ##Calculate the Kalman gain
        self.Xfiltered = self.Xpredicted + np.matmul(self.KGain, (Z - np.matmul(self.C, self.Xpredicted)))  ##Estimated updated state vector
        self.Pfiltered = self.Ppredicted - np.matmul(self.KGain, np.matmul(self.C, self.Ppredicted))    ##Estimated updated error co-variance matrix
        return (self.Xfiltered, self.Pfiltered)

class PIDController:
    def __init__(self, kp, ki, kd, target_distance, inverse=False):
        self.kp = kp
        self.ki = ki
        self.kd = kd
        self.target_distance = target_distance
        self.cumulative_error = 0
        self.last_error = 0
        self.last_time = rospy.Time.now()
        self.inverse = inverse
```

```
        self.inverse = inverse

    def compute(self, current_distance):
        if not self.inverse:
            error = self.target_distance - current_distance
        else:
            error = current_distance - self.target_distance
        current_time = rospy.Time.now()
        delta_time = (current_time - self.last_time).to_sec() if (current_time - self.last_time).to_sec() != 0 else 0.1

        self.cumulative_error += error * delta_time
        derivative = (error - self.last_error) / delta_time

        output = self.kp * error + self.ki * self.cumulative_error + self.kd * derivative

        self.last_error = error
        self.last_time = current_time

        return max(min(output, 1.0), 0.0)  # Limit output to between 0 and max allowable velocity

class VehicleController:
    def __init__(self, req_distance):
        self.pub_vel    = rospy.Publisher('/cmd_vel', Float64, queue_size=10, latch=True)
        self.pub_steer  = rospy.Publisher('/SteeringAngle', Float64, queue_size=10, latch=True)
        self.brk        = rospy.Publisher('/brakes', Float64, queue_size=10, latch=True)
        self.rate = rospy.Rate(100)

        self.req_distance = req_distance
        self.current_distance = 0
        self.current_orientation = 0
        self.brk.publish(0.0)
        self.pub_vel.publish(0.0)

        self.controllerSpeed = 1.0
        self.pedestrian_PID = PIDController(kp=0.0823, ki=0.000015, kd=0.45, target_distance=6.5, inverse=True)
        self.pedestrian_distance = queue.Queue()
        self.pedestrian_distance.put(500)

    def track_path(self):
        self.pid_straight = PIDController(kp=0.05, ki=0.000065, kd=0.045, target_distance=self.req_distance)
        while not rospy.is_shutdown():
            result = self.pid_straight.compute(self.current_distance)
            self.controllerSpeed = result
            self.publish_velocity()

            if self.current_distance >= self.req_distance-0.5:
                self.brk.publish(min(1-result, 0.5))
                if self.current_distance >= self.req_distance-0.2:
                    break

            self.rate.sleep()

        rospy.loginfo("Stopped")

    def update_odom_values(self, current_distance, current_orientation):
        self.current_distance = current_distance
        self.current_orientation = current_orientation

    def publish_velocity(self):
        controller_speed = 0.25*self.controllerSpeed
        detector_speed = self.pedestrian_PID.compute(list(self.pedestrian_distance.queue)[-1])  * 0.25
        rospy.loginfo(f"Controller Speed: {controller_speed}, Detector Speed: {detector_speed}")

        if controller_speed < detector_speed:
            self.brk.publish(0.0)
            self.pub_vel.publish(controller_speed)
        else:
            if detector_speed > 0.01:
                self.brk.publish(0.0)
                self.pub_vel.publish(detector_speed)
            else:
                self.brk.publish(1.0)
                self.pub_vel.publish(0.0)

class Odom_Manager:
    def __init__(self):
        self.last_z_orien = None
        self.orientation_multiplier = 0

    def odom_callback(self, odom_msg):
        position = odom_msg.pose.pose.position
        orientation = odom_msg.pose.pose.orientation

        noisy_position = np.array([[position.x], [position.y], [position.z]])
        noisy_orientation = np.array([[orientation.x], [orientation.y], [orientation.z]])

        Position_KF = Kalman_Filter(1.1345, 0.8245, 0.01)
        Position_KF.kf_prediction()    # Kalman filter prediction step
        X_filteredPos , P_filteredPos = Position_KF.kf_correction(noisy_position)    # Kalman filter correction step

        Position_KF.X = X_filteredPos
        Position_KF.P = P_filteredPos

        orientation_KF = Kalman_Filter(1.1345, 0.8245, 0.01)
        orientation_KF.kf_prediction()    # Kalman filter prediction step
        X_filteredOri , P_filteredOri = orientation_KF.kf_correction(noisy_orientation)    # Kalman filter correction step

        orientation_KF.X = X_filteredOri
        orientation_KF.P = P_filteredOri

        current_position = np.array([position.x, position.y])
        filtered_ori     = np.array((orientation.x, orientation.y, orientation.z, orientation.w))
        z_orien = euler_from_quaternion(filtered_ori)[2] * (180/math.pi)

        if z_orien < 0:
            z_orien = 360 + z_orien     # z_orien is a range from (0, 360)
        if self.last_z_orien is not None:
            if self.last_z_orien > 355 and z_orien < 5:
                self.orientation_multiplier += 1
            elif self.last_z_orien < 5 and z_orien > 355:
                self.orientation_multiplier -= 1

        total_z_orien = z_orien + (self.orientation_multiplier * 360)   # total_z_orien is a range from (-inf, inf)

        vCont.update_odom_values(current_position[1], total_z_orien)
        rospy.loginfo(f"z_ori: {z_orien}, total: {total_z_orien}")
        self.last_z_orien = z_orien     # last_z_orien is a range from (0, 360)

if __name__ == '__main__':
    DT = 0.1
    rospy.init_node('velocity_publisher', disable_signals=True)
    vCont = VehicleController(req_distance=75.0)

    detector = ObjectDetection()
    rospy.Subscriber("/image", Image, detector.detect)
    odom_class = Odom_Manager()
    rospy.Subscriber('/odom', Odometry, odom_class.odom_callback)

    vCont.track_path()
```

2. **LANE CHANGE TRACK:** Implementation: changes lanes when a close obstacle is detected. using a PID controller adjusting the steering angle, Challenges: lane changes without abrupt movement.

```python
#!/usr/bin/env python3

import rospy, numpy as np, cv2, torch, copy
from std_msgs.msg import Float64
from nav_msgs.msg import Odometry
from tf.transformations import euler_from_quaternion
import queue
import math
from cv_bridge import CvBridge
from ultralytics import YOLO
from sensor_msgs.msg import Image

class ObjectDetection:
    def __init__(self):
        self.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

        self.clss = ["car", "cone", "pedestrian"]
        self.model = YOLO("/home/morad/Downloads/F Rally/Milestone_1/f_ms/src/linear_track/src/CPC_model.pt")
        self.bridge = CvBridge()
        self.original_dict = {"car": [], "cone": [], "pedestrian": []}
        self.known_height_m = {"car": 1.55, "cone": 0.66, "pedestrian": 1.65}
        self.colors = [(238, 75, 43), (170, 255, 0), (0, 150, 255)]

        #track specific variables
        self.lane_change_ticket = False

    def detect(self, msg):
        self.detected = copy.deepcopy(self.original_dict)
        image = self.bridge.imgmsg_to_cv2(msg, "bgr8")
        results = self.model(image)
        boxes = results[0].boxes.cpu().numpy()  # Bounding boxes
        if len(boxes) > 0:
            for box in boxes:
                data = box.data[0]
                obj_name = self.clss[data[-1].astype(int)]
                confidence = data[-2].astype(float)
                x1, y1, x2, y2 = data[:4].astype(int)

                if confidence > 0.7:
                    object_in_shape = self.isInside(480, 240, 45, 480, 915, 480, x2 if x2 < 480 else x1, y2)
                    if object_in_shape:
                        distance = max((( (y2-y1) / self.known_height_m[obj_name] ) - 157.5) / -10.5), 0)
                        self.detected[obj_name].append(distance)
                    else:
                        distance = "N/A"
                    self.plot((image, data[-1].astype(int), (x1, y1, x2, y2), distance))
                else:
                    self.plot((image, None))
        else:
            self.plot((image, None))

        if self.detected["car"] and min(self.detected["car"]) <= 12.0 and self.lane_change_ticket == False:
            self.lane_change_ticket = True
            vCont.raise_change_lane_flag()
            self.lane_change_ticket = False

        if self.detected["pedestrian"]:
            vCont.pedestrian_distance.put(min(self.detected["pedestrian"]))
        else:
            vCont.pedestrian_distance.put(500)

    def plot(self, data):
        image = data[0]
        id = data[1]
        if id is not None:
            pos = data[2]
            try:
                dis = round(data[3], 2)
            except:
                dis = "N/A"
            cv2.rectangle(image, (pos[0], pos[1]), (pos[2], pos[3]), self.colors[id], 2)
            cv2.putText(image, f"dist: {dis}", (pos[0], pos[1]-5), cv2.FONT_ITALIC, 0.5, self.colors[id], 2)
        cv2.imshow("Image", image)
        cv2.waitKey(1)

    def isInside(self, x1, y1, x2, y2, x3, y3, x, y):
        # Calculate area of the main triangle
        A = abs(x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2)) / 2.0

        # Calculate area of the sub-triangles
        A1 = abs(x * (y2 - y3) + x2 * (y3 - y) + x3 * (y - y2)) / 2.0
        A2 = abs(x1 * (y - y3) + x * (y3 - y1) + x3 * (y1 - y)) / 2.0
        A3 = abs(x1 * (y2 - y) + x2 * (y - y1) + x * (y1 - y2)) / 2.0

        # Check if the point is inside the triangle
        return A == A1 + A2 + A3

class Kalman_Filter:
    def __init__(self, r1, r2, r3):
        # Define the Kalman filter parameters and variables
        self.dt = 0.1  # Time step
        self.A = np.array([[1, self.dt, 0, 0, 0, 0],
                           [0, 1, 0, 0, 0, 0],
                           [0, 0, 1, self.dt, 0, 0],
                           [0, 0, 0, 1, 0, 0],
                           [0, 0, 0, 0, 1, self.dt],
                           [0, 0, 0, 0, 0, 1]])  # State transition matrix
        self.B = np.zeros((6, 3))  # Control input matrix
        self.C = np.array([[1, 0, 0, 0, 0, 0],
                           [0, 0, 1, 0, 0, 0],
                           [0, 0, 0, 0, 1, 0]])  # Measurement matrix
        self.Q = np.eye(6) * 0.01  # Process noise covariance matrix
        self.R = np.array([[r1, 0, 0],
                           [0, r2, 0],
                           [0, 0, r3]])  # Measurement noise covariance matrix
        self.U = np.array([[0],[0.2],[0]])
        self.P = np.eye(6) * 10000  # Initial covariance matrix
        self.X = np.zeros((6, 1))  # Initial state estimate
        self.filtered_odom = []

        # Kalman filter prediction step
    def kf_prediction(self):
        self.Xpredicted = np.matmul(self.A, self.X) + np.matmul(self.B, self.U)        ##Predicted state vector                 (6x1)
        self.Ppredicted = np.matmul(self.A, np.matmul(self.P, np.transpose(self.A))) + self.Q   ##Predicted error co-variance matrix    (6x6)
        return (self.Xpredicted, self.Ppredicted)
    def kf_correction(self, Z):
        self.CTrans = np.transpose(self.C)
        self.num = np.matmul(self.Ppredicted, self.CTrans)      ##Numerature of Kalman gain equation
        self.den1 = np.matmul(self.C, self.Ppredicted)      ##CMatrix * PMatrix
        self.den2 = np.matmul(self.den1, self.CTrans) + self.R      ##CMatrix * PMatrix * CMatrix^T _+ R
        self.den = np.matrix(self.den2)      ##Place denumerator in matrix form
        self.deninv = self.den.getI()      ##(CMatrix * PMatrix * CMatrix^T _+ R) inverse
        self.KGain = np.matmul(self.num, self.deninv)      ##Calculate the Kalman gain
        self.Xfiltered = self.Xpredicted + np.matmul(self.KGain, (Z - np.matmul(self.C, self.Xpredicted)))  ##Estimated updated state vector
        self.Pfiltered = self.Ppredicted - np.matmul(self.KGain, np.matmul(self.C, self.Ppredicted))    ##Estimated updated error co-variance matrix
        return (self.Xfiltered, self.Pfiltered)

class PIDController:
    def __init__(self, kp, ki, kd, target_distance, inverse=False):
        self.kp = kp
        self.ki = ki
        self.kd = kd
        self.target_distance = target_distance
        self.cumulative_error = 0
        self.last_error = 0
        self.last_time = rospy.Time.now()
        self.inverse = inverse

    def compute(self, current_distance):
        if not self.inverse:
            error = self.target_distance - current_distance
        else:
            error = current_distance - self.target_distance
        current_time = rospy.Time.now()
        delta_time = (current_time - self.last_time).to_sec() if (current_time - self.last_time).to_sec() != 0 else 0.1

        self.cumulative_error += error * delta_time
        derivative = (error - self.last_error) / delta_time
```

```
        output = self.kp * error + self.ki * self.cumulative_error + self.kd * derivative

        self.last_error = error
        self.last_time = current_time

        return max(min(output, 1.0), 0.0)  # Limit output to between 0 and max allowable velocity

class VehicleController:
    def __init__(self, req_distance):
        self.pub_vel   = rospy.Publisher('/cmd_vel', Float64, queue_size=10, latch=True)
        self.pub_steer  = rospy.Publisher('/SteeringAngle', Float64, queue_size=10, latch=True)
        self.brk        = rospy.Publisher('/brakes', Float64, queue_size=10, latch=True)
        self.rate = rospy.Rate(100)

        self.req_distance = req_distance
        self.current_distance = 0
        self.current_orientation = 0
        self.brk.publish(0.0)
        self.pub_vel.publish(0.0)

        self.controllerSpeed = 1.0
        self.pedestrian_PID = PIDController(kp=0.0823, ki=0.000015, kd=0.45, target_distance=6.5, inverse=True)
        self.pedestrian_distance = queue.Queue()
        self.pedestrian_distance.put(500)

        self.change_lane_flag = False

    def track_path(self):
        self.pid_straight = PIDController(kp=0.05, ki=0.000065, kd=0.045, target_distance=self.req_distance)
        while not rospy.is_shutdown():
            result = self.pid_straight.compute(self.current_distance)
            self.controllerSpeed = result
            self.publish_velocity()

            if self.change_lane_flag:
                self.Change_lane('left')
                self.Change_lane('right')
                self.change_lane_flag = False

            if self.current_distance >= self.req_distance-0.3:
                self.brk.publish(min(1-result, 0.5))
                rospy.loginfo("Stopped")
                if self.current_distance >= self.req_distance-0.2:
                    break

            self.rate.sleep()

    def raise_change_lane_flag(self):
        self.change_lane_flag = true

    def update_odom_values(self, current_distance, current_orientation):
        self.current_distance = current_distance
        self.current_orientation = current_orientation

    def Change_lane(self, lane, angle = 32):
        max_steering_angle = 30
        if lane == 'left':
            Target_angle = self.current_orientation + angle
            inverse = False
            const = 1
        elif lane == 'right':
            Target_angle = self.current_orientation - angle
            inverse = True
            const = -1
        else:
            raise ValueError("Invalid lane value")

        self.pid_lane_change = PIDController(kp=0.0375, ki=0.001, kd=0.0675, target_distance=Target_angle, inverse=inverse)
        rospy.loginfo(($self.current_orientation < Target_angle) if not inverse else ($self.current_orientation > Target_angle))
        while ($self.current_orientation < Target_angle) if not inverse else ($self.current_orientation > Target_angle):
            self.publish_velocity()
            steering_angle = self.pid_lane_change.compute(self.current_orientation)*max_steering_angle
            self.pub_steer.publish(steering_angle * const)
            rospy.loginfo(f"{lane} turn: {self.current_orientation}, {Target_angle} -- {steering_angle}")
            self.rate.sleep()
        self.pub_steer.publish(0)

    def publish_velocity(self):
        controller_speed = 0.22*self.controllerSpeed
        detector_speed = self.pedestrian_PID.compute(list(self.pedestrian_distance.queue)[-1])  * 0.22
        rospy.loginfo(f"Controller Speed: {controller_speed/0.22}, Detector Speed: {detector_speed/0.22}")

        if controller_speed < detector_speed:
            self.brk.publish(0.0)
            self.pub_vel.publish(controller_speed)
        else:
            if detector_speed > 0.01:
                self.brk.publish(0.0)
                self.pub_vel.publish(detector_speed)
            else:
                self.brk.publish(1.0)
                self.pub_vel.publish(0.0)

class Odom_Manager:
    def __init__(self):
        self.last_z_orien = None
        self.orientation_multiplier = 0

    def odom_callback(self, odom_msg):
        position = odom_msg.pose.pose.position
        orientation = odom_msg.pose.pose.orientation

        noisy_position = np.array([[position.x], [position.y], [position.z]])
        noisy_orientation = np.array([[orientation.x], [orientation.y], [orientation.z]])

        Position_KF = Kalman_Filter(1.1345, 0.8245, 0.01)
        Position_KF.kf_prediction()     # Kalman filter prediction step
        X_filteredPos , P_filteredPos = Position_KF.kf_correction(noisy_position)     # Kalman filter correction step

        Position_KF.X = X_filteredPos
        Position_KF.P = P_filteredPos

        orientation_KF = Kalman_Filter(1.1345, 0.8245, 0.01)
        orientation_KF.kf_prediction()     # Kalman filter prediction step
        X_filteredOri , P_filteredOri = orientation_KF.kf_correction(noisy_orientation)     # Kalman filter correction step

        orientation_KF.X = X_filteredOri
        orientation_KF.P = P_filteredOri

        current_position = np.array([position.x, position.y])
        filtered_ori     = np.array((orientation.x, orientation.y, orientation.z, orientation.w))
        z_orien = euler_from_quaternion(filtered_ori)[2] * (180/math.pi)

        if z_orien < 0:
            z_orien = 360 + z_orien      # z_orien is a range from (0, 360)
        if self.last_z_orien is not None:
            if self.last_z_orien > 355 and z_orien < 5:
                self.orientation_multiplier += 1
            elif self.last_z_orien < 5 and z_orien > 355:
                self.orientation_multiplier -= 1

        total_z_orien = z_orien + (self.orientation_multiplier * 360)   # total_z_orien is a range from (-inf, inf)

        vCont.update_odom_values(current_position[1], total_z_orien)
        rospy.loginfo(f"z_ori: {z_orien}, total: {total_z_orien}")
        self.last_z_orien = z_orien      # last_z_orien is a range from (0, 360)

if __name__ == '__main__':
    DT = 0.1
    rospy.init_node('velocity_publisher', disable_signals=True)
    vCont = VehicleController(req_distance=75.0)

    detector = ObjectDetection()
    rospy.Subscriber("/image", Image, detector.detect)
    odom_class = Odom_Manager()
    rospy.Subscriber('/odom', Odometry, odom_class.odom_callback)

    vCont.track_path()
```

3. **CIRCULAR TRACK:** The car is following a circular track, which can be a useful test for evaluating the performance of the PID controller and other navigation algorithms.

```python
#!/usr/bin/env python3

import rospy, numpy as np, cv2, torch, copy
from std_msgs.msg import Float64
from nav_msgs.msg import Odometry
from tf.transformations import euler_from_quaternion
import queue
import math
from cv_bridge import CvBridge
from ultralytics import YOLO
from sensor_msgs.msg import Image

class ObjectDetection:
    def __init__(self):
        self.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

        self.ciss = ["car", "cone", "pedestrian"]
        self.model = YOLO("/home/morad/Downloads/E-Rally/Milestone 1/f ms/src/linear track/src/CPC model.pt")
        self.bridge = CvBridge()
        self.original_dict = {"car": [], "cone": [], "pedestrian": []}
        self.known_height_m = {"car": 1.45, "cone": 0.66, "pedestrian": 1.65}
        self.colors = [(238, 75, 43), (170, 255, 0), (0, 150, 255)]

        #track specific variables
        self.avoid_collision_ticket = False

    def detect(self, msg):
        self.detected = copy.deepcopy(self.original_dict)
        image = self.bridge.imgmsg_to_cv2(msg, "bgr8")
        results = self.model(image)
        boxes = results[0].boxes.cpu().numpy()  # Bounding boxes
        if len(boxes) > 0:
            for box in boxes:
                data = box.data[0]
                obj_name = self.ciss[data[-1].astype(int)]
                confidence = data[-2].astype(float)
                x1, y1, x2, y2 = data[:4].astype(int)

                if confidence > 0.7:
                    # object_in_shape = self.isinside(4300, 240, 45, 4300, 915, 4300, x2 if x2 < 4300 else x1, y2)
                    # if object_in_shape:
                    distance = max(((( (y2 y1) / self.known_height_m[obj_name] )  - 157.5) /  10.5), 0)
                    self.detected[obj_name].append(distance)
                    # else:
                    #     distance = "N/A"
                    self.plot((image, data[-1].astype(int), (x1, y1, x2, y2), distance))
                else:
                    self.plot((image, None))
        else:
            self.plot((image, None))

        if self.detected["cone"] and min(self.detected["cone"]) <= 9:
            self.avoid_collision_ticket = True
            vCont.raise_avoid_collision_flag()
            self.avoid_collision_ticket = False

        if self.detected["pedestrian"]:
            vCont.pedestrian_distance.put(min(self.detected["pedestrian"]))
        else:
            vCont.pedestrian_distance.put(5000)

    def plot(self, data):
        image = data[0]
        id = data[1]
        if id is not None:
            pos = data[2]
            try:
                dis = round(data[3], 2)
            except:
                dis = "N/A"
            cv2.rectangle(image, (pos[0], pos[1]), (pos[2], pos[3]), self.colors[id], 2)
            cv2.putText(image, f"dist: {dis}", (pos[0], pos[1]-5), cv2.FONT_ITALIC, 0.5, self.colors[id], 2)
        cv2.imshow("Image", image)
        cv2.waitKey(1)

    def isInside(self, x1, y1, x2, y2, x3, y3, x, y):
        # Calculate area of the main triangle
        A = abs(x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2)) / 2.0

        # Calculate area of the sub triangles
        A1 = abs(x * (y2 - y3) + x2 * (y3 - y) + x3 * (y - y2)) / 2.0
        A2 = abs(x1 * (y - y3) + x * (y3 - y1) + x3 * (y1 - y)) / 2.0
        A3 = abs(x1 * (y2 - y) + x2 * (y - y1) + x * (y1 - y2)) / 2.0

        # Check if the point is inside the triangle
        return A == A1 + A2 + A3

class Kalman_Filter:
    def __init__(self, r1, r2, r3):
        # Define the Kalman filter parameters and variables
        self.dt = 0.1  # Time step
        self.A = np.array([[1, self.dt, 0, 0, 0, 0],
                           [0, 1, 0, 0, 0, 0],
                           [0, 0, 1, self.dt, 0, 0],
                           [0, 0, 0, 1, 0, 0],
                           [0, 0, 0, 0, 1, self.dt],
                           [0, 0, 0, 0, 0, 1]])  # State transition matrix
        self.B = np.zeros((6, 4))  # Control input matrix
        self.C = np.array([[1, 0, 0, 0, 0, 0],
                           [0, 0, 1, 0, 0, 0],
                           [0, 0, 0, 0, 1, 0]])  # Measurement matrix
        self.Q = np.eye(6) * 0.01  # Process noise covariance matrix
        self.R = np.array([[r1, 0, 0],
                           [0, r2, 0],
                           [0, 0, r3]])  # Measurement noise covariance matrix
        self.U = np.array([[0],[0.2],[0]])
        self.P = np.eye(6) * 10000  # Initial covariance matrix
        self.X = np.zeros((6, 1))  # Initial state estimate
        self.filtered_odom = []

        # Kalman filter prediction step
    def kf_prediction(self):
        self.Xpredicted = np.matmul(self.A, self.X) + np.matmul(self.B, self.U)          ##Predicted state vector                                (6x1)
        self.Ppredicted = np.matmul(self.A, np.matmul(self.P, np.transpose(self.A))) + self.Q   ##Predicted error co-variance matrix     (6x6)
        return (self.Xpredicted, self.Ppredicted)
    def kf_correction(self, Z):
        self.CTrans = np.transpose(self.C)
        self.num = np.matmul(self.Ppredicted, self.CTrans)      ##Numerature of Kalman gain equation
        self.den1 = np.matmul(self.C, self.Ppredicted)    ##CMatrix * PMatrix
        self.den2 = np.matmul(self.den1, self.CTrans) + self.R      ##CMatrix * PMatrix * CMatrix^T _+ R
        self.den = np.matrix(self.den2)                  ##Place denemrature in matrix form
        self.deninv = self.den.getI()                    ##(CMatrix * PMatrix * CMatrix^T  + R) Inverse
        self.KGain = np.matmul(self.num, self.deninv)      ##Calculate the Kalman gain
        self.Xfiltered = self.Xpredicted + np.matmul(self.KGain, (Z - np.matmul(self.C, self.Xpredicted)))  ##Estimated updated state vector
        self.Pfiltered = self.Ppredicted - np.matmul(self.KGain, np.matmul(self.C, self.Ppredicted))    ##Estimated updated error co-variance matrix
        return (self.Xfiltered, self.Pfiltered)

class PIDController:
    def __init__(self, kp, ki, kd, target_distance, inverse=False):
        self.kp = kp
        self.ki = ki
        self.kd = kd
        self.target_distance = target_distance
        self.cumulative_error = 0
        self.last_error = 0
        self.last_time = rospy.Time.now()
        self.inverse = inverse

    def compute(self, current_distance):
        if not self.inverse:
            error = self.target_distance - current_distance
        else:
            error = current_distance - self.target_distance
        current_time = rospy.Time.now()
        delta_time = (current_time - self.last_time).to_sec() if (current_time - self.last_time).to_sec() != 0 else 0.1

        self.cumulative_error += error * delta_time
        derivative = (error - self.last_error) / delta_time

        output = self.kp * error + self.ki * self.cumulative_error + self.kd * derivative

        self.last_error = error
        self.last_time = current_time
```

```
        self.last_error = error
        self.last_time = current_time

        return max(min(output, 1.0), 0.0)  # Limit output to between 0 and max allowable velocity

class VehicleController:
    def __init__(self, req_distance):
        self.pub_vel    = rospy.Publisher('/cmd_vel', Float64, queue_size=10, latch=True)
        self.pub_steer  = rospy.Publisher('/SteeringAngle', Float64, queue_size=10, latch=True)
        self.brk        = rospy.Publisher('/brakes', Float64, queue_size=10, latch=True)
        self.rate = rospy.Rate(100)

        self.req_distance = req_distance
        self.current_distance = 0
        self.current_orientation = 0
        self.brk.publish(0.0)
        self.pub_vel.publish(0.0)

        self.controllerSpeed = 1.0
        self.pedestrian_PID = PIDController(kp=0.0823, ki=0.000002, kd=0.343, target_distance=8.5, inverse=True)
        self.pedestrian_distance = queue.Queue()
        self.pedestrian_distance.put(500)

        self.avoid_collision_flag = False
        self.chosen_angle = 5


    def track_path(self):
        self.pid_straight = PIDController(kp=0.05, ki=0.000065, kd=0.045, target_distance=self.req_distance)
        self.pub_steer.publish(self.chosen_angle)
        while not rospy.is_shutdown():
            result = self.pid_straight.compute(self.current_orientation)
            self.controllerSpeed = result
            self.publish_velocity()

            if self.avoid_collision_flag:
                self.steering = self.Steer_Vehicle('right', 5)
                self.steering = self.Steer_Vehicle('left', 60)
                self.steering = self.Steer_Vehicle('right', 5)
                self.avoid_collision_flag = false

            if self.current_orientation >= self.req_distance-2:
                self.brk.publish(min(1-result, 0.8))
                rospy.loginfo("Stopped")
                if self.current_orientation >= self.req_distance:
                    break

            self.rate.sleep()

    def raise_avoid_collision_flag(self):
        self.avoid_collision_flag = True

    def update_odom_values(self, current_distance, current_orientation):
        self.current_distance = current_distance
        self.current_orientation = current_orientation

    def Steer_Vehicle(self, lane, angle = 33):
        max_steering_angle = 30
        if lane == 'left':
            Target_angle = self.current_orientation + angle
            inverse = False
            const = 1
        elif lane == 'right':
            Target_angle = self.current_orientation - angle
            inverse = True
            const = -1
        else:
            raise ValueError("Invalid lane value")

        self.pid_lane_change = PIDController(kp=0.0375, ki=0.001, kd=0.0675, target_distance=Target_angle, inverse=inverse)
        rospy.loginfo((self.current_orientation < Target_angle) if not inverse else (self.current_orientation > Target_angle))
        while (self.current_orientation < Target_angle) if not inverse else (self.current_orientation > Target_angle):
            self.publish_velocity()
            steering_angle = self.pid_lane_change.compute(self.current_orientation)*max_steering_angle
            self.pub_steer.publish(steering_angle * const)
            rospy.loginfo(f"{lane} turn: {self.current_orientation}, {Target_angle} -- {steering_angle}")
            self.rate.sleep()
        self.pub_steer.publish(self.chosen_angle)

    def publish_velocity(self):
        controller_speed = 0.25*self.controllerSpeed
        detector_speed = self.pedestrian_PID.compute(list(self.pedestrian_distance.queue)[-1])  * 0.25
        rospy.loginfo(f"Controller Speed: {controller_speed/0.25}, Detector Speed: {detector_speed/0.25}")

        if controller_speed < detector_speed:
            self.brk.publish(0.0)
            self.pub_vel.publish(controller_speed)
        else:
            if detector_speed > 0.01:
                self.brk.publish(0.0)
                self.pub_vel.publish(detector_speed)
            else:
                self.brk.publish(1.0)
                self.pub_vel.publish(0.0)

class Odom_Manager:
    def __init__(self):
        self.last_z_orien = None
        self.orientation_multiplier = 0

    def odom_callback(self, odom_msg):
        position = odom_msg.pose.pose.position
        orientation = odom_msg.pose.pose.orientation

        noisy_position = np.array([[position.x], [position.y], [position.z]])
        noisy_orientation = np.array([[orientation.x], [orientation.y], [orientation.z]])

        Position_KF = Kalman_Filter(1.1345, 0.8245, 0.01)
        Position_KF.kf_prediction()     # Kalman filter prediction step
        X_filteredPos , P_filteredPos = Position_KF.kf_correction(noisy_position)     # Kalman filter correction step

        Position_KF.X = X_filteredPos
        Position_KF.P = P_filteredPos

        orientation_KF = Kalman_Filter(1.1345, 0.8245, 0.01)
        orientation_KF.kf_prediction()     # Kalman filter prediction step
        X_filteredOri , P_filteredOri = orientation_KF.kf_correction(noisy_orientation)     # Kalman filter correction step

        orientation_KF.X = X_filteredOri
        orientation_KF.P = P_filteredOri

        current_position = np.array([position.x, position.y])
        #filtered_ori     = np.array((orientation.x, orientation.y, orientation.z, orientation.w))
        z_orien = euler_from_quaternion(filtered_ori)[2] * (180/math.pi)

        if z_orien < 0:
            z_orien = 360 + z_orien     # z_orien is a range from (0, 360)
        if self.last_z_orien is not None:
            if self.last_z_orien > 355 and z_orien < 5:
                self.orientation_multiplier += 1
            elif self.last_z_orien < 5 and z_orien > 355:
                self.orientation_multiplier -= 1
            elif self.last_z_orien - z_orien > 180:
                self.orientation_multiplier += 1

        total_z_orien = z_orien + (self.orientation_multiplier * 360)   # total_z_orien is a range from (-inf, inf)

        vCont.update_odom_values(current_position[1], total_z_orien)
        rospy.loginfo(f"z_ori: {z_orien}, total: {total_z_orien}")
        self.last_z_orien = z_orien     # last_z_orien is a range from (0, 360)

if __name__ == '__main__':
    DT = 0.1
    rospy.init_node('velocity_publisher', disable_signals=True)
    vCont = VehicleController(req_distance=720)

    detector = ObjectDetection()
    rospy.Subscriber("/image", Image, detector.detect)
    odom_class = Odom_Manager()
    rospy.Subscriber('/odom', Odometry, odom_class.odom_callback)

    vCont.track_path()
```

Challenges in LINEAR TRACK, LANE CHANGE TRACK, CIRCULAR TRACK: involves integrating real-time object detection with decision-making. As we use YOLO to detect obstacles and vehicles then use this information for adjusting steering, speed, and lane changes accordingly. Adapting to each scenario showcases our system's robustness in autonomous navigation.

4. **CUSTOM TRACK:** This track is designed to include:
    - Different corners to test the accuracy of steering angles.
    - Features various obstacles such as:
        - Another car: to test lane changes.
        - Cones: to evaluate car maneuverability around objects.
        - Pedestrians: to test its response, as the vehicle stops until the pedestrian moves safely to the other side of the road then continues moving.

So, the track is specifically tailored to test the car's responsiveness to dynamic environments and suddenly appearing obstacles, necessitating real-time adjustments in speed and direction.

Challenges in CUSTOM TRACK: training the object detection model (*custom_c7_model.pt*) to handle dynamic situations, ensuring the vehicle can make decisions in a timely manner.

Custom track code:

```
#!/usr/bin/env python3

import rospy, numpy as np, cv2, torch, copy, time
from std_msgs.msg import Float64
from nav_msgs.msg import Odometry
from tf.transformations import euler_from_quaternion
import threading, queue
import math
from cv_bridge import CvBridge, CvBridgeError
from ultralytics import YOLO
from sensor_msgs.msg import Image

class ObjectDetection:
    def __init__(self):
        self.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

        self.clss = ["banner",
                     "building",
                     "car",
                     "cone",
                     "grass",
                     "pavement",
                     "pedestrian", ]
        self.model = YOLO("/home/morad/Downloads/E-Rally/Milestone_1/f_ms/src/linear_track/src/custom_c7_model.pt")
        # self.model.fuse()
        self.bridge = CvBridge()
        self.original_dict = {"car": [], "cone": [], "pedestrian": []}
        self.known_height_m = {"car": 1.5%, "cone": 0.66, "pedestrian": 1.6%}
        self.colors = [", ", ", (29%, 7%, 4%), (170, 29%, 0), ", ", (0, 14%, 25%)]

        self.override = False
        #track specific variables
        self.avoid_collision_ticket = False
        self.lane_change_ticket = False

    def detect(self, msg):
        self.detected = copy.deepcopy(self.original_dict)
        image = self.bridge.imgmsg_to_cv2(msg, "bgr8")
        results = self.model(image)
        boxes = results[0].boxes.cpu().numpy()  # Bounding boxes
        if len(boxes) > 0:
            for box in boxes:
                data = box.data[0]
                id = data[-1].astype(int)
                obj_name = self.clss[id]
                confidence = data[-2].astype(float)
                x1, y1, x2, y2 = data[:4].astype(int)

                if id in (2, 3, 6) and confidence > 0.7:
                    object_in_shape = self.isInside(480, 240, 45, 480, 915, 480, x2 if x2 < 480 else x1, y2)
                    if object_in_shape or self.override:
                        distance = max(((( (y2-y1) / self.known_height_m[obj_name] ) - 157.5) / -10.5), 0)
                        self.detected[obj_name].append(distance)
                    else:
                        distance = "N/A"
                    self.plot((image, id, (x1, y1, x2, y2), distance))
                else:
                    self.plot((image, None))
        else:
            self.plot((image, None))

        #? for each object in the detected dictionary, check if the minimum number (distance) is less than a specific distance. if so, activate the corresponding action.
        if self.detected["car"] and min(self.detected["car"]) <= 12.0 and self.lane_change_ticket == False:
            self.lane_change_ticket = True
            vCont.raise_change_lane_flag()
            self.lane_change_ticket = False

        if self.detected["cone"] and min(self.detected["cone"]) <= 9.0 and self.avoid_collision_ticket == False:
            self.avoid_collision_ticket = True
            vCont.raise_avoid_collision_flag()
            self.avoid_collision_ticket = False

        if self.detected["pedestrian"]:
            vCont.pedestrian_distance.put(min(self.detected["pedestrian"]))
        else:
            vCont.pedestrian_distance.put(500)

    def plot(self, data):
        image = data[0]
        id = data[1]
        if id is not None:
            pos = data[2]
            try:
                dis = round(data[3], 2)
            except:
                dis = "N/A"
            cv2.rectangle(image, (pos[0], pos[1]), (pos[2], pos[3]), self.colors[id], 2)
            cv2.putText(image, f"dist: {dis}", (pos[0], pos[1] ), cv2.FONT_ITALIC, 0.5, self.colors[id], 2)
        cv2.imshow("image", image)
        cv2.waitKey(1)

    def isInside(self, x1, y1, x2, y2, x3, y3, x, y):
        # Calculate area of the main triangle
        A = abs(x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2)) / 2.0

        # Calculate area of the sub-triangles
        A1 = abs(x * (y2 - y3) + x2 * (y3 - y) + x3 * (y - y2)) / 2.0
        A2 = abs(x1 * (y - y3) + x * (y3 - y1) + x3 * (y1 - y)) / 2.0
        A3 = abs(x1 * (y2 - y) + x2 * (y - y1) + x * (y1 - y2)) / 2.0

        # Check if the point is inside the triangle
        return A == A1 + A2 + A3

class Kalman_Filter:
    def __init__(self, r1, r2, r3):
        # Define the Kalman filter parameters and variables
        self.dt = 0.1  # Time step
        self.A = np.array([[1, self.dt, 0, 0, 0, 0],
                           [0, 1, 0, 0, 0, 0],
                           [0, 0, 1, self.dt, 0, 0],
                           [0, 0, 0, 1, 0, 0],
                           [0, 0, 0, 0, 1, self.dt],
                           [0, 0, 0, 0, 0, 1]])  # State transition matrix
        self.B = np.zeros((6, 3))  # Control input matrix
        self.C = np.array([[1, 0, 0, 0, 0, 0],
                           [0, 0, 1, 0, 0, 0],
                           [0, 0, 0, 0, 1, 0]])  # Measurement matrix
        self.Q = np.eye(6) * 0.01  # Process noise covariance matrix
        self.R = np.array([[r1, 0, 0],
                           [0, r2, 0],
                           [0, 0, r3]])  # Measurement noise covariance matrix
        self.U = np.array([[0],[0.2],[0]])
        self.P = np.eye(6) * 10000  # Initial covariance matrix
        self.X = np.zeros((6, 1))  # Initial state estimate
        self.filtered_odom = []

        # Kalman filter prediction step
    def KF_prediction(self):
        self.Xpredicted = np.matmul(self.A, self.X) + np.matmul(self.B, self.U)        ##Predicted state vector                      (6x1)
        self.Ppredicted = np.matmul(self.A, np.matmul(self.P, np.transpose(self.A))) + self.Q   ##Predicted error co variance matrix    (6x6)
        return (self.Xpredicted, self.Ppredicted)
    def kf_correction(self, z):
        self.CTrans = np.transpose(self.C)
        self.num = np.matmul(self.Ppredicted, self.CTrans)      ##Numerature of Kalman gain equation
        self.den1 = np.matmul(self.C, self.Ppredicted)      ##CMatrix * PMatrix
        self.den2 = np.matmul(self.den1, self.CTrans) + self.R      ##CMatrix * PMatrix * CMatrix^T _+ R
        self.den = np.matrix(self.den2)             ##Place denemarature in matrix form
        self.denInv = self.den.getI()               ##(CMatrix * PMatrix * CMatrix^T _+ R) Inverse
        self.KGain = np.matmul(self.num, self.denInv)           ##Calculate the Kalman gain
        self.Xfiltered = self.Xpredicted + np.matmul(self.KGain, (z - np.matmul(self.C, self.Xpredicted)))  ##Estimated updated state vector
        self.Pfiltered = self.Ppredicted - np.matmul(self.KGain, np.matmul(self.C, self.Ppredicted))   ##Estimated updated error co variance matrix
        return (self.Xfiltered, self.Pfiltered)

class PIDController:
    def __init__(self, kp, ki, kd, target_distance, inverse=False):
        self.kp = kp
        self.ki = ki
        self.kd = kd
        self.target_distance = target_distance
        self.cumulative_error = 0
        self.last_error = 0
        self.last_time = rospy.Time.now()
        self.inverse = inverse

    def compute(self, current_distance):
        if not self.inverse:
            error = self.target_distance - current_distance
        else:
            error = current_distance - self.target_distance
        current_time = rospy.Time.now()
        delta_time = (current_time - self.last_time).to_sec() if (current_time - self.last_time).to_sec() != 0 else 0.1

        self.cumulative_error += error * delta_time
        derivative = (error - self.last_error) / delta_time

        output = self.kp * error + self.ki * self.cumulative_error + self.kd * derivative

        self.last_error = error
        self.last_time = current_time

        return max(min(output, 1.0), 0.0)  # Limit output to between 0 and max allowable velocity

class VehicleController:
    def __init__(self, req_distance):
        self.pub_vel   = rospy.Publisher('/cmd_vel', Float64, queue_size=10, latch=True)
        self.pub_steer  = rospy.Publisher('/SteeringAngle', Float64, queue_size=10, latch=True)
        self.brk       = rospy.Publisher('/brakes', Float64, queue_size=10, latch=True)
        self.rate = rospy.Rate(100)

        self.req_distance = req_distance
        self.current_distance = None
        self.current_x = 0    # x
        self.current_y = 0    # y
```

```python
        self.current_distance = None
        self.current_x = 0    # x
        self.current_y = 0    # y
        self.current_orientation = 0    # z
        self.brk.publish(0.0)
        self.pub_vel.publish(0.0)

        self.controllerSpeed = 1.0
        self.pedestrian_PTD = PIDController(kp=0.083%, ki=0.000001%, kd=0.4%, target_distance=6.5, inverse=True)  #kp=0.083%, ki=0.000002, kd=0.34%
        self.pedestrian_distance = queue.Queue()
        self.pedestrian_distance.put(500)

        self.change_lane_flag = False
        self.cl_working = False
        self.avoid_collision_flag = False
        self.ac_working = False

        self.key_1 = 1
        self.key_2 = 1
        self.key_3 = 1
        self.key_4 = 1
        self.key_5 = 0

    def track_path(self):
        self.pid_straight = PIDController(kp=0.05, ki=0.00005, kd=0.045, target_distance=self.req_distance)
        while not rospy.is_shutdown():
            result = self.pid_straight.compute(self.current_x)
            self.controllerSpeed = result
            self.publish_velocity()

            if self.key_1 and self.current_y >= 70.0:
                self.key_1 = 0
                self.Steer_Vehicle('right', 90, 20)
            elif self.key_2 and self.current_x >= 75.0:
                self.key_2 = 0
                detector.override = True
                self.Steer_Vehicle('right', 180, 7)
                detector.override = False
            elif self.key_3 and self.key_2==0 and self.current_x <= 35.0:
                self.key_3 = 0
                self.Steer_Vehicle('left', 90, 20)
            elif self.key_4 and self.key_3==0 and self.current_y <= 5:
                self.key_4 = 0
                self.key_5 = 1
                self.Steer_Vehicle('right', 90, 20)
                self.pid_straight.target_distance = 0
                self.pid_straight.inverse = True
            elif self.key_5 and self.current_x < 7:
                self.brk.publish(min(1-result, 0.8))
                if self.current_x < 0:
                    rospy.loginfo("Stopped")
                    rospy.signal_shutdown("Track Completed")

            self.rate.sleep()

    def raise_change_lane_flag(self):
        if not self.cl_working:
            self.change_lane_flag = True

    def raise_avoid_collision_flag(self):
        if not self.ac_working:
            self.avoid_collision_flag = True

    def check_flags(self):
        if self.change_lane_flag:
            self.change_lane_flag = False
            self.cl_working = True
            self.Steer_Vehicle('left')
            self.Steer_Vehicle('right')
            self.cl_working = False

        if self.avoid_collision_flag:
            self.avoid_collision_flag = False
            self.ac_working = True
            self.Steer_Vehicle('left', 3, state="collision")
            self.Steer_Vehicle('right', 80, state="collision")  # this angle is related to the radius of the track
            self.Steer_Vehicle('left', 5, state="collision")
            self.ac_working = False

    def update_odom_values(self, current_distance, current_orientation):
        self.current_distance = current_distance
        self.current_x = self.current_distance[0]
        self.current_y = self.current_distance[1]
        self.current_orientation = current_orientation

    def Steer_Vehicle(self, lane, angle = 33, max_st=30, state="normal"):
        max_steering_angle = max_st
        if lane == 'left':
            Target_angle = self.current_orientation + angle
            inverse = False
            const = 1
        elif lane == 'right':
            Target_angle = self.current_orientation - angle
            inverse = True
            const = -1
        else:
            raise ValueError("invalid lane value")

        if state == "collision":
            pid_lane_change = PIDController(kp=0.045, ki=0.001, kd=0.06, target_distance=Target_angle, inverse=inverse)
        elif state == "normal":
            pid_lane_change = PIDController(kp=0.0375, ki=0.001, kd=0.0675, target_distance=Target_angle, inverse=inverse)
        rospy.loginfo((self.current_orientation < Target_angle) if not inverse else (self.current_orientation > Target_angle))
        while (self.current_orientation < Target_angle) if not inverse else (self.current_orientation > Target_angle):
            self.publish_velocity()
            steering_angle = pid_lane_change.compute(self.current_orientation)*max_steering_angle
            self.pub_steer.publish(steering_angle * const)
            rospy.loginfo(f"{lane} turn <{state}>: {self.current_orientation}, {Target_angle} -- {steering_angle}")
            self.rate.sleep()
        self.pub_steer.publish(0)

    def publish_velocity(self):
        controller_speed = 0.25*self.controllerSpeed
        detector_speed = self.pedestrian_PTD.compute(list(self.pedestrian_distance.queue)[-1])  * 0.25
        rospy.loginfo(f"Controller Speed: {controller_speed/0.25}, Detector Speed: {detector_speed/0.25}")
        # Centeral Speed Publisher
        if controller_speed < detector_speed:
            self.brk.publish(0.0)  # Release Brakes
            self.pub_vel.publish(controller_speed)
        else:
            if detector_speed > 0.01:
                self.brk.publish(0.0)  # Release brakes
                self.pub_vel.publish(detector_speed)  # Publish speed command
            else:
                self.brk.publish(1.0)  # Apply brakes
                self.pub_vel.publish(0.0)  # Stop the car

        self.check_flags()

class Odom_Manager:
    def __init__(self):
        self.last_z_orien = None
        self.orientation_multiplier = 0

    def odom_callback(self, odom_msg):
        position = odom_msg.pose.pose.position
        orientation = odom_msg.pose.pose.orientation

        noisy_position = np.array([[position.x], [position.y], [position.z]])
        noisy_orientation = np.array([[orientation.x], [orientation.y], [orientation.z]])

        Position_KF = Kalman_Filter(1.1345, 0.8245, 0.01)
        Position_KF.kf_prediction()    # Kalman filter prediction step
        X_filteredPos , P_filteredPos = Position_KF.kf_correction(noisy_position)    # Kalman filter correction step

        Position_KF.X = X_filteredPos
        Position_KF.P = P_filteredPos

        orientation_KF = Kalman_Filter(1.1345, 0.8345, 0.01)
        orientation_KF.kf_prediction()    # Kalman filter prediction step
        X_filteredOri , P_filteredOri = orientation_KF.kf_correction(noisy_orientation)    # Kalman filter correction step

        orientation_KF.X = X_filteredOri
        orientation_KF.P = P_filteredOri

        current_position = np.array([position.x, position.y])
        filtered_ori   = np.array([orientation.x, orientation.y, orientation.z, orientation.w])
        z_orien = euler_from_quaternion(filtered_ori)[2] * (180/math.pi)

        if z_orien < 0:
            z_orien = 360 + z_orien     # z_orien is a range from (0, 360)
        if self.last_z_orien is not None:
            if self.last_z_orien > 355 and z_orien < 5:
                self.orientation_multiplier += 1
            elif self.last_z_orien < 5 and z_orien > 355:
                self.orientation_multiplier -= 1

        total_z_orien = z_orien + (self.orientation_multiplier * 360)   # total z orien is a range from (-inf, inf)

        vCont.update_odom_values((current_position[0], current_position[1]), total_z_orien)
        rospy.loginfo(f"Z_ori: {z_orien}, total: {total_z_orien}")
        self.last_z_orien = z_orien     # last_z_orien is a range from (0, 360)

if __name__ == '__main__':
    DT = 0.1
    rospy.init_node('velocity_publisher', disable_signals=True)
    vCont = VehicleController(req_distance=150.0)

    detector = ObjectDetection()
    rospy.Subscriber("/image", Image, detector.detect)
    odom_class = Odom_Manager()
    rospy.Subscriber('/odom', Odometry, odom_class.odom_callback)

    vCont.track_path()
```

All the previously mentioned tracks utilize object detection to respond to obstacles and pedestrians, ensuring they are avoided before continuing along the desired path. This approach ensures safe autonomous driving.

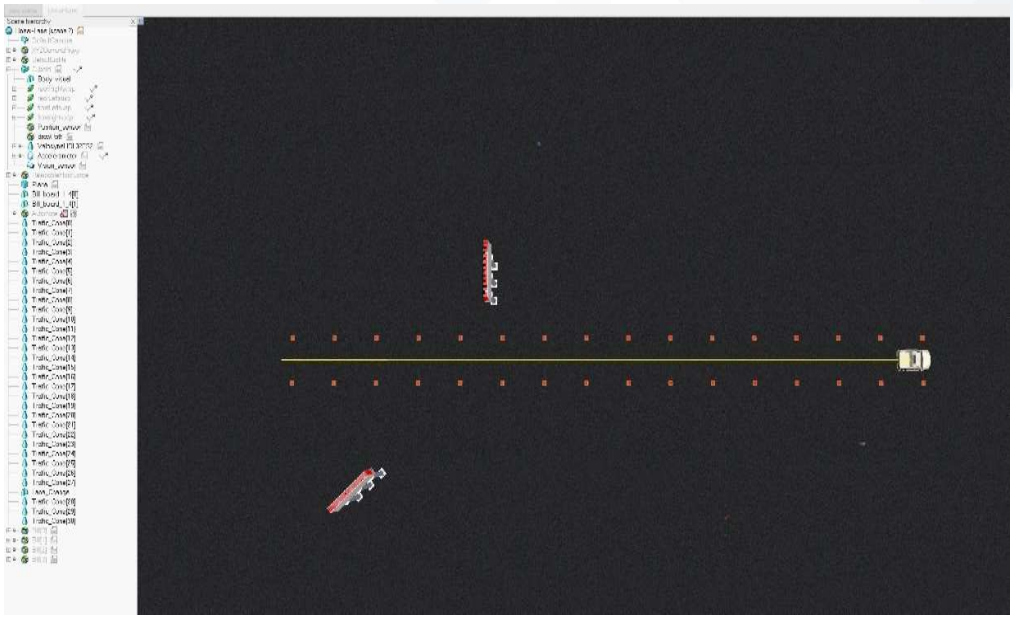
Algorithm Description

LINEAR TRACK, LANE CHANGE TRACK, CIRCULAR TRACK & CUSTOM TRACK:

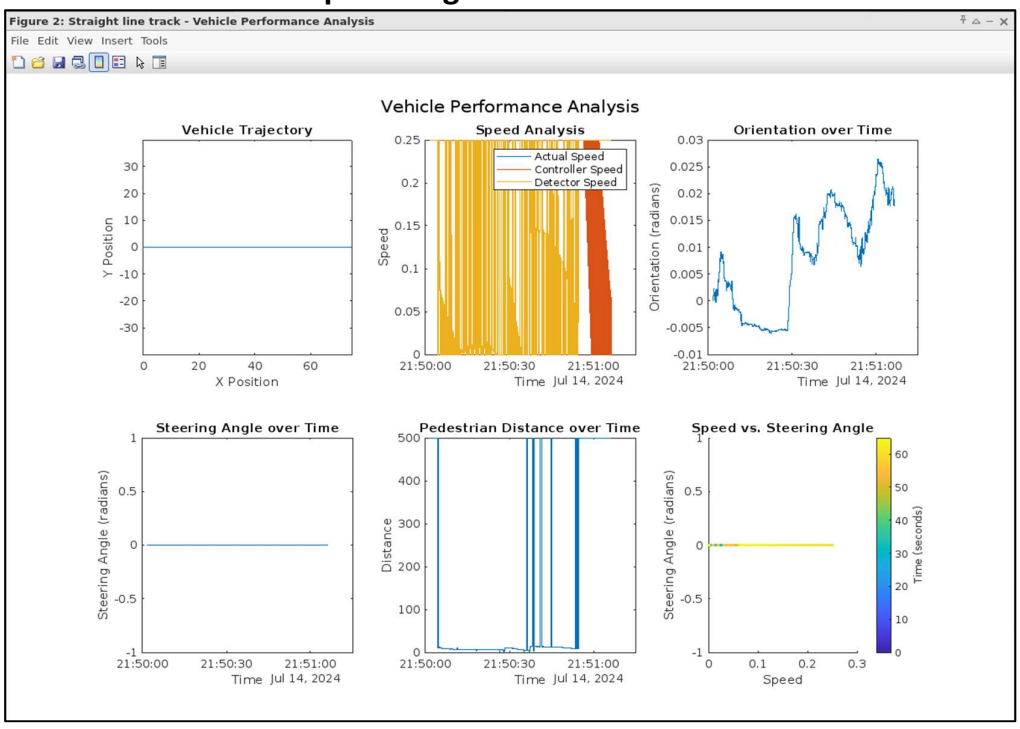
- **Object Detection:** Enhanced YOLO model by filtering detections based on a confidence threshold and using known object dimensions for accurate distance estimation.
- **Kalman Filter**: Tuned process and measurement noise covariances for optimal performance in filtering noisy odometry data.
- **PID Control**: Adjusted PID gains through trial and error to ensure smooth speed transitions and accurate distance maintenance

Performance analysis

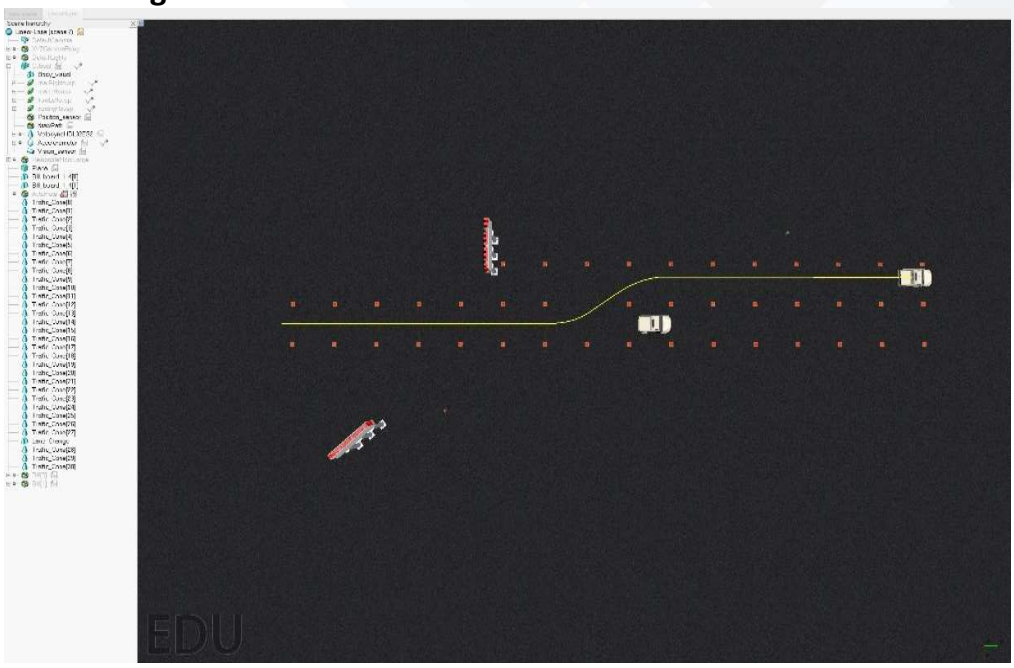
1. Linear track:



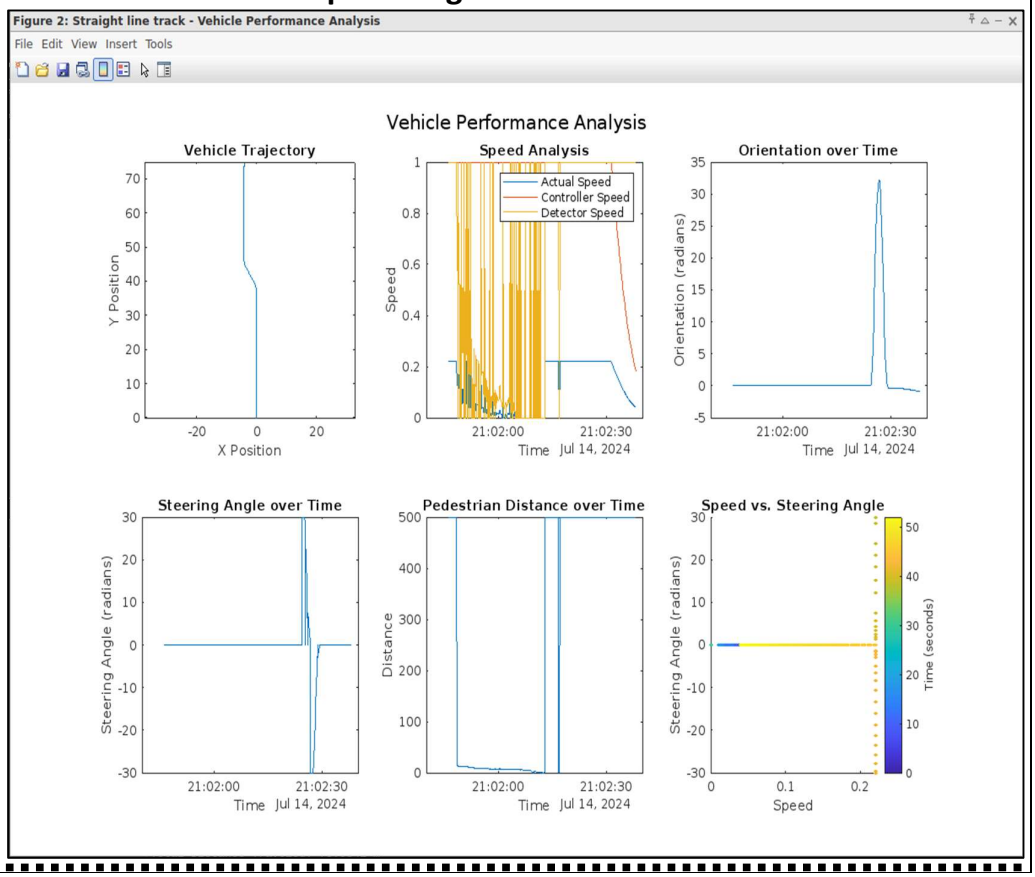
Graphs using MATLAB:



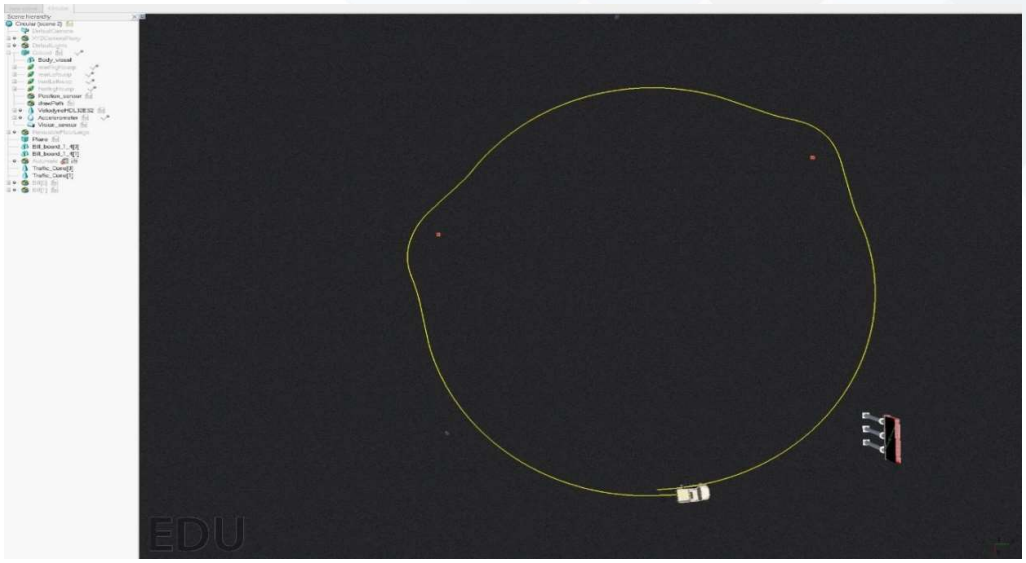
2. Lane change track:



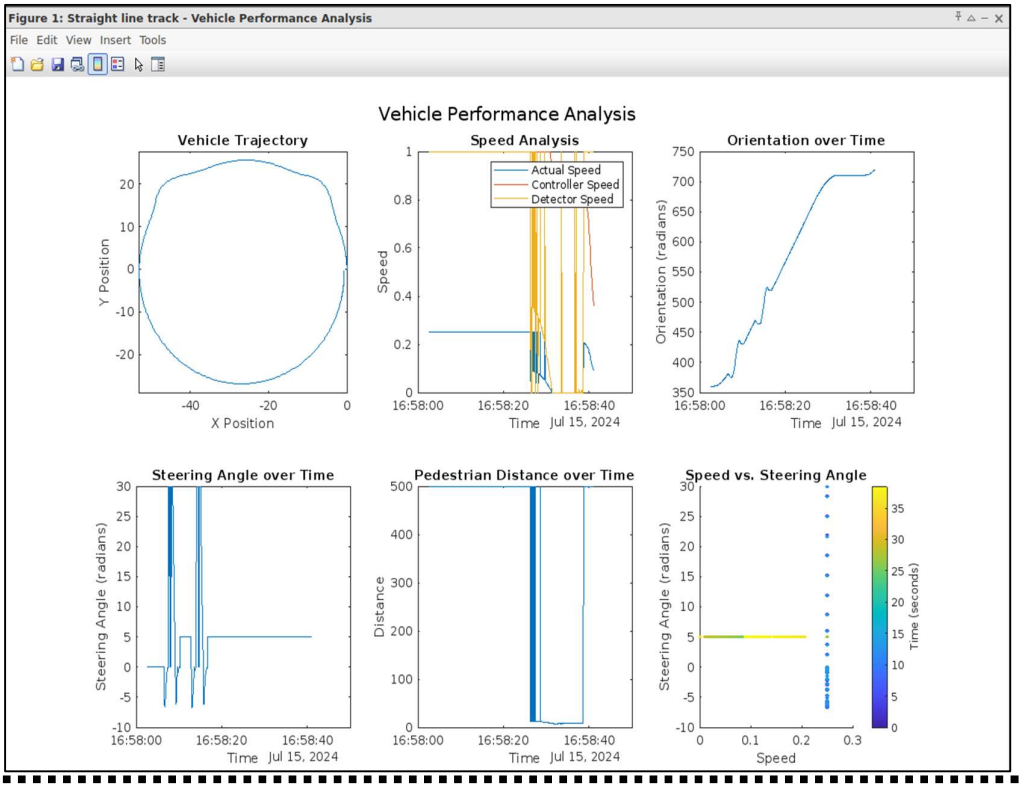
Graphs using MATLAB:



3. Circular Track:

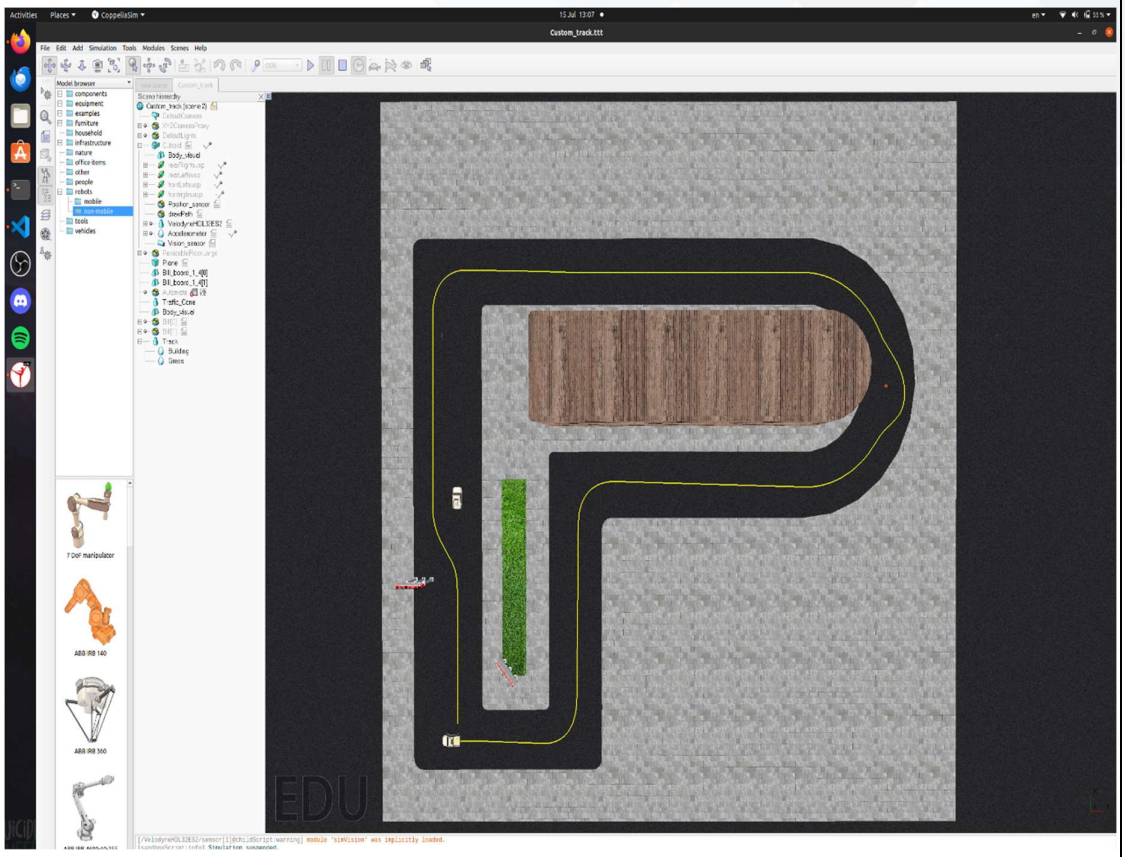


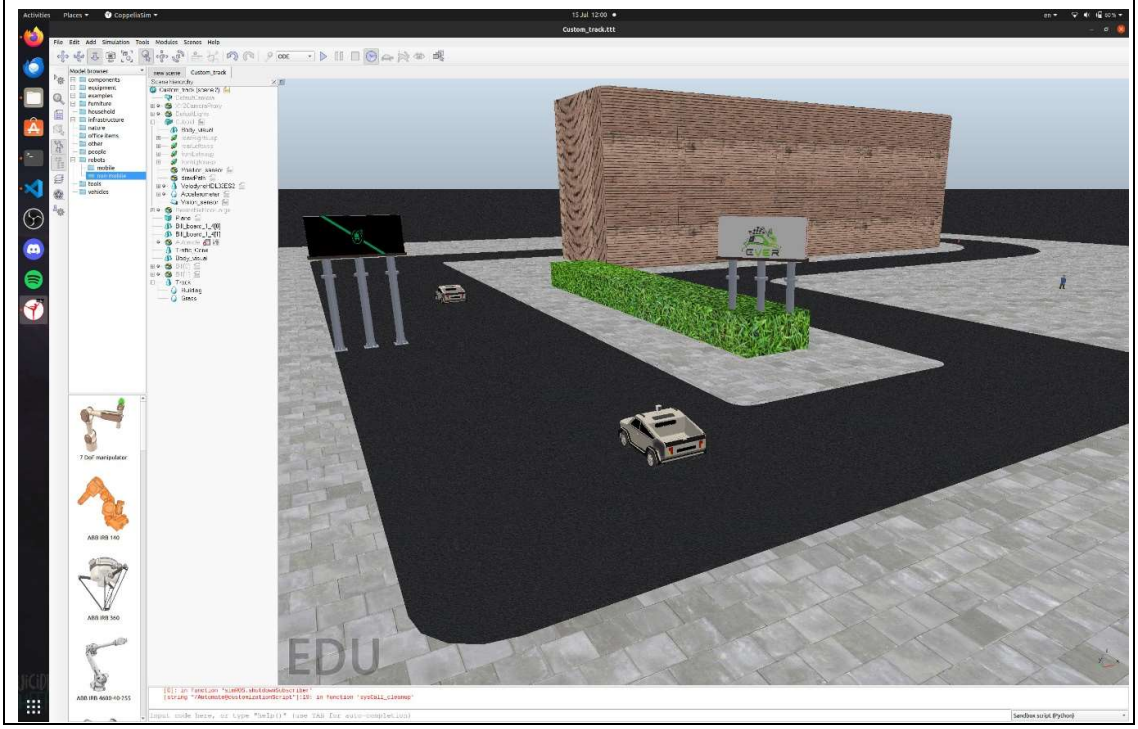
Graphs using MATLAB:

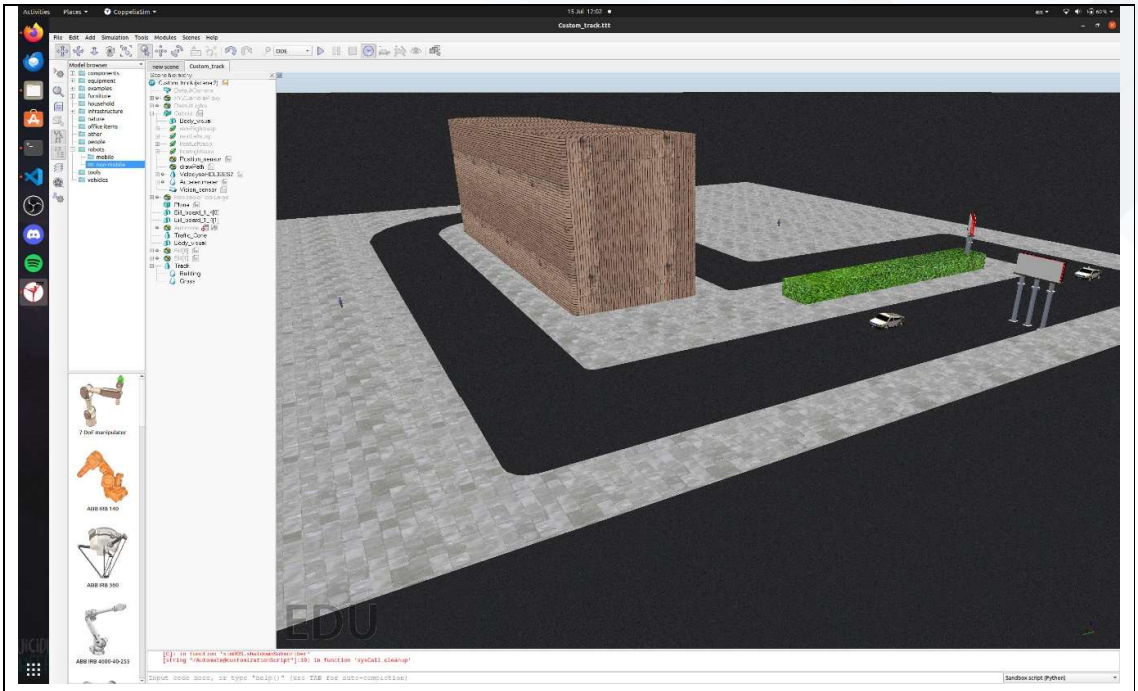


Challenges in LINEAR TRACK, LANE CHANGE TRACK, CIRCULAR TRACK: Integrating the object detection module with the vehicle controller module posed challenges in effectively using detection data for real-time decision-making. We addressed optimizing algorithms for swift data processing and testing across varied track scenarios. This integration ensures our system can autonomously navigate easily on the different tracks.

4. Custom track:



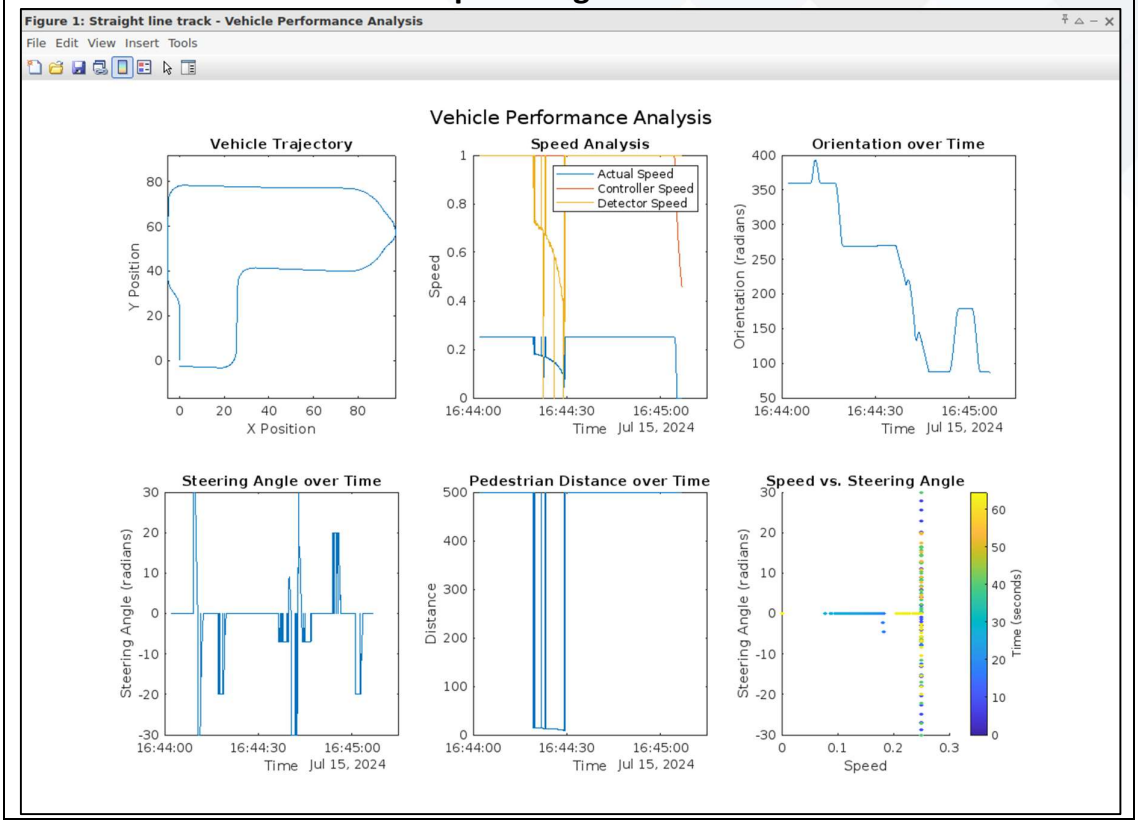






Challenges in CUSTOM TRACK: include training the old object detection model to handle dynamic situations, ensuring the vehicle can make decisions in a timely manner.

Graphs using MATLAB:



Conclusion

This project represents a major advancement in our autonomous vehicle simulation endeavor. By combining YOLO for real-time object detection, Kalman filtering & PID controllers for accurate trajectory tracking and speed control, we've successfully integrated key learnings from earlier milestones. This integration enables our vehicle to navigate various scenarios with enhanced precision, adaptability, and reliability.